

48 – [Sql Server](#)

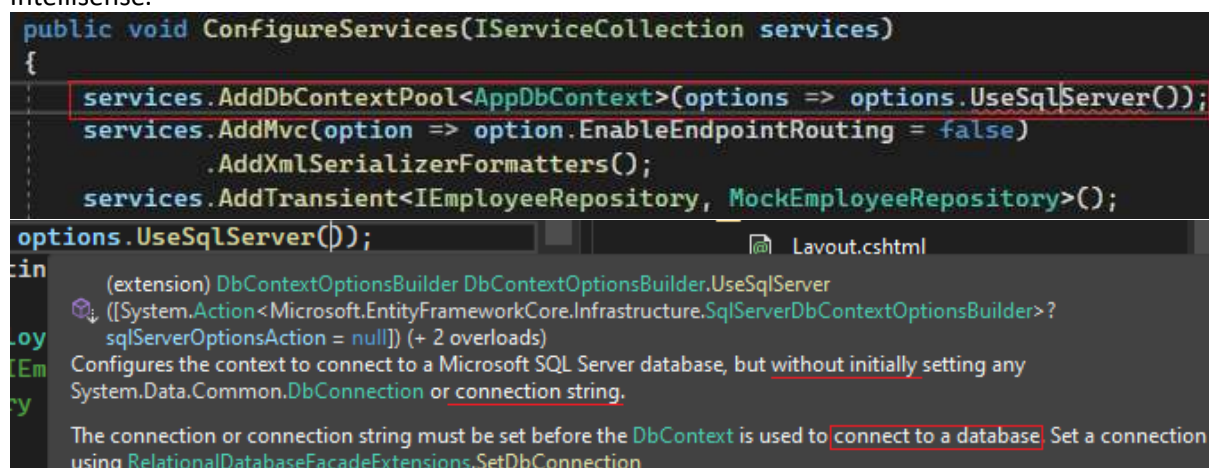
Искаме да използваме MSSqlServer with EntityFrameworkCore. Това става чрез ConfigureServices – Startup.cs

Миналия урок създадохме AppDbContext class, който трябва да регистрираме чрез Dependency injection in Startup.cs.

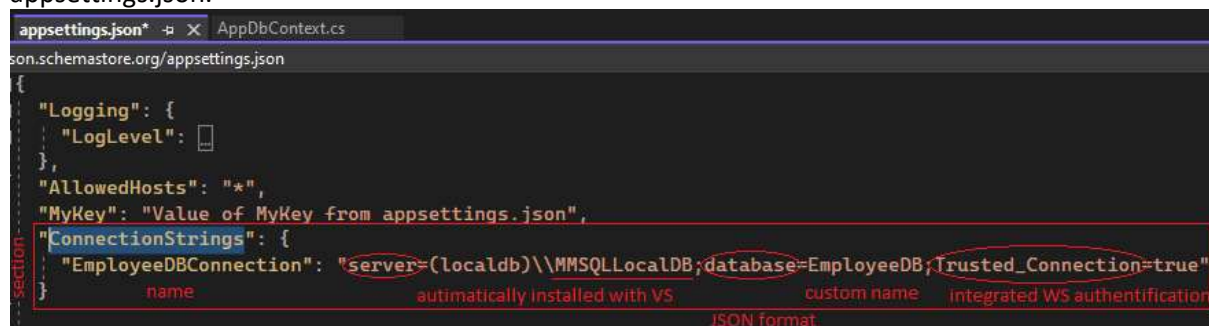


With **DbContext** pooling, an instance from the **DbContext** pool is provided if available, rather than creating a new instance.

Всеки път когато инстанция на DbContext е заявена, вместо да се създаде нова инстанция, Asp.NetCore проверява дали тя съществува в DbContextPool и ако е налична, връща нея. От гледна точка на производителност, AddDbContextPool() е по-добър от AddDbContext(). След това трябва да вмъкнем необходимия provider със съответното namespace, за да се включи и Intellisense.



Вместо да вложим този connection string вътре в UseSqlServer(), нека използваме appsettings.json.



В класически Asp.Net, съхраняваме конфигурационна информация в web.config файл, който е в XML формат. В Asp.Net.Core обаче, имаме различни конфигурационни източници, един от които е appsettings.json, който ги съхранява в JSON формат.

EmployeeDB още не е създадена.

```
"ConnectionStrings": {
```

"EmployeeDBConnection":

```
"server=(localdb)\\MSSQLLocalDB;database=EmployeeDB;Trusted_Connection=true"
```

Сега трябва да го укажем в IConfiguration:

```
private IConfiguration _config; // auto generated private field;

0 references
public Startup(IConfiguration config) //+ auto MS namespace, parameter named config[...]

// This method gets called by the runtime. Use this method to add services to the container. ...
0 references
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(
        options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));
    // name

    services.AddMvc(option => option.EnableEndpointRouting = false)
        .AddXmlSerializerFormatters();
    services.AddTransient<IEmployeeRepository, MockEmployeeRepository>();

    services.AddDbContextPool<AppDbContext>(
        options =>
options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));
}
```

Using SQL Server with EF Core

```
public class Startup
{
    private IConfiguration _config;

    public Startup(IConfiguration config)
    {
        _config = config;
    }

    public void ConfigureServices(IServiceCollection services)
    {
        services.AddDbContextPool<AppDbContext>(
            options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));

        services.AddMvc().AddXmlSerializerFormatters();
        services.AddTransient<IEmployeeRepository, MockEmployeeRepository>();
    }

    // Rest of the code
}
```

What is the difference

```
"server=(localdb)\\MSSQLLocalDB;database=EmployeeDB;Trusted_Connection=true"
```

```
Trusted_Connection=true
Integrated Security=SSPI
Integrated Security=true
```

- All the 3 settings specify the same thing
- Use Integrated Windows Authentication instead SQL Server authentication

Към момента, нашето приложение все още използва този hard code in-memory list:

```
MockEmployeeRepository.cs  AppDbContext.cs
EmployeeManagement.Models.MockEmployeeRepository  _employeeList

namespace EmployeeManagement.Models
{
    2 references
    public class MockEmployeeRepository : IEmployeeRepository
    {
        private List<Employee> _employeeList; // field
        0 references
        public MockEmployeeRepository() // ctor
        {
            _employeeList = new List<Employee>()
            {
                new Employee() { Id = 1, Name = "Mary", Department = Dept.HR, Email = "Mary@abv.bg" },
                new Employee() { Id = 2, Name = "John", Department = Dept.IT, Email = "John@abv.bg" },
                new Employee() { Id = 3, Name = "Sam", Department = Dept.IT, Email = "Sam@abv.bg" }
            };
        }
    }
}
```

В следващото видео ще направим SqlServer repository, от което могат да се извличат и обработват данни чрез local server DB.